AI-POWERED SE-MAX • ROUND 1 IDEA PROPOSAL

Port0

Switch tools. Keep the context. Leave the secrets behind.

Team: **Hallucinati**
Rohan Saeed & Shayan Ahmed

1. ROUND 1 SUBMISSION SUMMARY

Port0 is a local, read-only command-line tool that captures the working state of a git repository (branch, staged/unstaged/untracked changes, failing test outputs, and task intent) into a single deterministic, secret-redacted, size-bounded handoff packet. Designed for AI-native software workflows, Port0 allows developers to transition seamlessly between local AI agents (e.g., Claude Code), web-hosted interfaces (e.g., Claude.ai), or human teammates when hitting rate limits or session crashes—without ever re-explaining the project or leaking sensitive environment credentials.

2. PROBLEM STATEMENT & MARKET NEED

Software engineers using AI coding agents routinely move work between terminal CLI tools, IDE extensions, web chat interfaces, and teammates. Each transition causes severe friction: developers waste 10–15 minutes re-describing the task, copy-pasting diffs, hunting for failing test logs, and manually selecting modified files. Crucially, manual copy-pasting frequently leads to accidental leakage of `.env` files, API tokens, and private keys into third-party web chats. Port0 solves this universal developer headache by packaging in-progress workspace state instantly, safely, and predictably.

3. THE FOUR ROUND 1 APPROVAL CRITERIA

Port0 directly satisfies all four official approval tests for Round 1 entry:

Approval Test	Proposal Evaluation & Compliance Justification
1. Feasibility	**PASS:** Highly scoped, single-purpose TypeScript CLI utility with zero complex cloud dependencies, database servers, or paid APIs. Fully buildable, testable, and deployable within the 6-hour hackathon window by two strong coders.
2. Usefulness	**PASS:** Solves an immediate, industrial-scale pain point affecting millions of developers adopting AI agents daily: eliminating downtime caused by context loss and preventing sensitive credential leaks during tool switches.
3. Originality	**PASS:** Unlike generic repository packers that dump entire codebases into prompts, Port0 focuses strictly on *in-progress working changes* with mathematical guarantees: deterministic byte ordering, bounded size limits, and multi-pattern secret redaction.
4. Ethics & Safety	**PASS:** Completely read-only, local, and offline tool. Executes zero remote commands, opens no network tunnels, collects zero personal data, and actively protects user security by stripping credentials before export.

4. SYSTEM CONCEPT & CORE ARCHITECTURE

Port0 operates strictly as a local Node.js CLI (`npx port0 export`). It collects current repository state via safe `git` subprocess calls, applies rigorous path and pattern filters, and serializes a clean Markdown or JSON payload.

Core Component	Design & Functionality Overview
Git State Collector	Reads active branch, HEAD commit, staged/unstaged diffs, untracked files, and file renames by executing `git` with fixed parameters and isolated environment variables.
Path Guard	Resolves real canonical paths to ensure all read operations remain strictly inside the project root directory, blocking path traversal and malicious symlink escapes.
Secret Redactor	Applies a path denylist (e.g., `.env*`, `*.pem`, `id_rsa`) and content pattern filters (cloud keys, tokens, JWTs), followed by a fail-closed final scan.
Size Budgeter	Enforces strict per-section and total byte caps (default 200 KB) with explicit truncation markers to prevent prompt context overflows.
Deterministic Serializer	Sorts file paths, fixes key order, normalizes line endings (LF), and computes a SHA-256 content hash to guarantee byte-identical output for identical repository states.

5. HIGH-LEVEL SCOPE BOUNDARIES

In-Scope for Build Day	Strict Non-Goals (Out of Scope)
• Single CLI command: `port0 export`. • Comprehensive git working tree state parsing. • Secret redaction & strict root path containment. • Bounded size budgeter with truncation bounds. • Deterministic byte serializer with SHA-256 hash. • Clean exit codes & robust error handling.	• **NO** arbitrary command or test execution. • **NO** writing or modifying project source files. • **NO** network listeners, tunnels, or remote access. • **NO** cloud databases, accounts, or telemetry. • **NO** unconstrained LLM decision-making.

6. TECHNOLOGY STACK

Language & Runtime: TypeScript on Node.js / Bun

Validation & Testing: Zod (type-safe validation schemas), Vitest (unit testing harness), fast-check (property-based string testing)

Process Execution: Execa (safe child-process calls to local `git` CLI)